# C# & MySQL Veritabanı İşlemleri Kapsamlı Rehberi

ADO.NET Mimarisi, Sorgu Tipleri, Güvenlik (SQL Injection) ve Pratik Kod Şablonları

🗄️ **Kapsam:** MySqlConnection, MySqlCommand, DataReader, DataAdapter, DataTable 🛡️ **Güvenlik:** Parametreli Sorgular, using Blokları, İstisna Yönetimi

💡 **Hedef:** Öğrenciler ve Başlangıç-Orta Seviye Geliştiriciler

## 1. ADO.NET Nedir ve Nasıl Çalışır? (Bileşen Mimarisi)

**ADO.NET**, .NET platformunda C# programlarının veritabanları (MySQL, MSSQL, Oracle, PostgreSQL vb.) ile iletişim kurmasını sağlayan resmi veri erişim kütüphanesidir. PHP'deki **PDO** veya **mysqli** yapısının C#'taki kurumsal karşılığıdır.

**1. MySqlConnection** (Kablo / Bağlantı Yolu) ➔ **2. MySqlCommand** (Kurye / SQL Emri) ➔ **3. Çalıştırma Metodu** (NonQuery / Scalar / Reader) ➔ **4. Sonuç / Nesne** (Etkilenen Sayı / Veri Akışı)

🍽️ **Günlük Hayat Benzetmesi (Kiler ve Kurye Modeli):**

- **MySqlConnection (Telefon Hattı):** Mutfak ile bodrumdaki kiler (MySQL) arasında kurulan sesli telefon hattıdır. Hat açılmadan (Open) sipariş verilemez.
- **MySqlCommand (Sipariş Fişi / Kurye):** Kasiyerin elindeki yazılı emirdir: "*Depodan 5 çuval un getir*" veya "*Raf 3'teki peyniri sil*".
- **MySqlDataReader (Yürüyen Bant):** Kilere inip malları tek tek, hızlıca yukarı çıkaran tek yönlü taşıma bandıdır (Hafiftir, hızlıdır).
- **MySqlDataAdapter & DataTable (Alışveriş Sepeti):** Kilerdeki tüm malları toptan kasaya doldurup masaya koyan çevrimdışı tepsidir (DataGridView için idealdir).

## 2. Veritabanı Bağlantısı Nasıl Oluşturulur? (`MySqlConnection`)

C#'ta bir MySQL sunucusuna bağlanabilmek için projenize **`MySql.Data`** (veya modern **`MySqlConnector`**) NuGet paketinin ekli olması gerekir. Bağlantının ilk şartı doğru bir **Connection String (Bağlantı Cümlesi)** tanımlamaktır.

### A) Connection String Parametreleri ve Anlamları

| Parametre | Örnek Değer | Açıklama |
|---|---|---|
| **`Server`** | `localhost` veya `127.0.0.1` | Veritabanı sunucusunun IP adresi veya alan adı. |
| **`Database`** | `teknik_servis` | Bağlanılacak veritabanının adı. |
| **`Uid`** (User Id) | `teknik_servis_user` / `root` | Veritabanı kullanıcısının adı. |
| **`Pwd`** (Password) | `123456` (root için genelde boş) | Veritabanı kullanıcısının parolası. |
| **`Port`** | `3306` (Varsayılandır) | MySQL'in dinlediği port numarası (Standart dışı portlarda zorunludur). |
| **`Charset`** | `utf8mb4` | Türkçe karakterlerin (ş, ğ, ı, ö, ü, ç) ve emojilerin bozulmasını önler. |

**Standart Bağlantı Cümlesi Formatı:**

```
string connStr = "Server=localhost;Database=teknik_servis;Uid=teknik_servis_user;Pwd=123456;Charset=utf8mb4;";
```

### B) Bağlantıyı Yönetmenin Altın Kuralı: `using` ve Bağlantı Havuzu (Connection Pool)

Veritabanı bağlantıları işletim sisteminde ve sunucuda yüksek maliyetli kaynaklardır. `conn.Open()` yapıldıktan sonra kodun içinde bir hata fırlatılırsa (Exception), alt satırdaki `conn.Close()` koduna asla ulaşılamaz! Bağlantı sunucuda açık asılı kalır ve bir süre sonra **"Too many connections" (Bağlantı havuzu doldu)** hatasıyla sistem kilitlenir.

⚠️ **ÇÖZÜM: `using` Bloğu ile Otomatik Kapatma!**
Bağlantıyı daima `using (MySqlConnection conn = ...) { ... }` bloğu içinde açın. Blok bittiği an (hata çıksa dahi) C# arka planda `conn.Dispose()` çalıştırır ve bağlantıyı havuza güvenle iade eder.

## 3. SQL Komutu (`MySqlCommand`), Parametreler ve `@` İşareti

`MySqlCommand`, veritabanı üzerinde çalıştırılacak SQL sorgusunu ve bu sorgunun hangi bağlantı (`conn`) üzerinden gönderileceğini belirleyen sınıftır.

> 🚨 **SQL INJECTION TEHLİKESİ — Metin Birleştirme (`+`) Kesinlikle Yasaktır!**
> `cmd.CommandText = "SELECT * FROM calisanlar WHERE tc = '" + txt_tc.Text + "'";`
> Kötü niyetli bir kullanıcı TC kutusuna `' OR '1'='1` yazdığında SQL sorgusu tüm sistemi savunmasız bırakır, şifreleri açığa çıkarır veya tabloları silebilir!

> 🛡️ **Parametreli Sorguların (`@parametre`) Çalışma Mantığı:**
> Sorgu içerisine kullanıcı verisini doğrudan yapıştırmak yerine **`@tc`, `@ad`** gibi yer tutucular (parametreler) koyarız. MySQL bu parametreleri çalıştırılabilir SQL komutu olarak değil, **saf veri (düz metin)** olarak ele alır. Kullanıcı içine zararlı SQL kodu yazsa dahi komut çalışmaz; sadece metin olarak saklanır.

```
MySqlCommand cmd = new MySqlCommand("INSERT INTO calisanlar (ad, tc) VALUES (@ad, @tc)", conn);
cmd.Parameters.AddWithValue("@ad", calisan.Ad);
cmd.Parameters.AddWithValue("@tc", calisan.Tc);
```

> 💡 **SIK SORULAN SORU: Sorgulardaki İki Farklı `@` İşareti Neden Vardır?**
>
> Yeni başlayanların en çok karıştırdığı şey, tırnak başındaki `@` ile SQL içindeki `@` işaretinin aynı şey sanılmasıdır. İkisi tamamen farklı dünyalara aittir:
>
> **1. Tırnağın Başındaki `@` (`@"..."` - Verbatim String):**
> **C# diline aittir.** Normal çift tırnakta Enter'a basıp alt satıra geçemezsiniz. Tırnağın başına `@` koyduğunuzda uzun SQL sorgularını alt alta satırlara bölüp ferahça yazabilirsiniz. Ayrıca dosya yollarındaki ters slash (`\`) kaçışlarını devre dışı bırakır (`@"C:\resimler\foto.jpg"`).
>
> **2. SQL İçindeki `@` (`@ad`, `@tc` - SQL Parametresi):**
> **MySQL veritabanına aittir.** Kullanıcıdan gelen veriyi SQL komutu olarak değil, güvenli bir değer (veri kutusu) olarak sorguya bağlar. SQL Injection saldırılarını %100 engelleyen emniyet kemeridir.

## 4. Hangi Sorgu İçin Hangi Sınıf ve Metot Kullanılır? (Altın Başvuru Tablosu)

| Metot / Sınıf | Uygun SQL Komutları | Döndürdüğü Değer | Kullanım Amacı ve Örneği |
|---|---|---|---|
| **`ExecuteNonQuery()`**<br>(MySqlCommand) | `INSERT`<br>`UPDATE`<br>`DELETE` | **`int`**<br>(Etkilenen satır sayısı) | Veritabanında değişiklik yapar ama satır okumaz. Geriye `1` dönerse işlem başarılıdır, `0` ise hiçbir satır etkilenmemiştir.<br>*Örn: Yeni çalışan kaydetme, şifre güncelleme.* |
| **`ExecuteScalar()`**<br>(MySqlCommand) | `SELECT COUNT(*)`<br>`SELECT MAX(id)`<br>`SELECT SUM(maas)` | **`object`**<br>(İlk satırın ilk hücresi) | Sorgudan tek bir kutucuk/değer döneceği zaman kullanılır. Sonuç `object` döner, uygun tipe dönüştürülür (`Convert.ToInt32` vb.).<br>*Örn: "Bu TC kayıtlı mı?" kontrolü.* |
| **`ExecuteReader()`**<br>(MySqlDataReader) | `SELECT`<br>(Çok satır ve çok sütun) | **`MySqlDataReader`**<br>(Hızlı okuma akışı) | Veritabanından satır satır hızlı ve düşük bellek tüketimiyle veri okur. `while(dr.Read())` döngüsüyle model nesnelerine doldurulur.<br>*Örn: Çalışanlar listesini `List<Calisan>` yapmak.* |
| **`MySqlDataAdapter`**<br>& `DataTable` | `SELECT`<br>(Toptan tablo çekme) | **`DataTable`**<br>(Hafızadaki tablo) | Çevrimdışı (Disconnected) çalışır. Bağlantıyı kendisi açar, verileri `DataTable`'a doldurur ve hemen kapatır. WinForms `DataGridView` için en pratik yoldur. |

## 5. Veritabanına Veri Yazma: Ekleme, Güncelleme ve Silme (CUD)

Veritabanında kayıt oluşturan, güncelleyen veya silen tüm işlemlerde **`ExecuteNonQuery()`** metodu kullanılır.

### A) Yeni Kayıt Ekleme (`INSERT` + `ExecuteNonQuery`)

```
public bool CalisanEkle(Calisan calisan, out string hataMesaji)
{
    hataMesaji = "";
    // 💡 Baştaki '@' işareti sayesinde sorguyu alt alta ferahça yazabiliyoruz:
    string sql = @"INSERT INTO calisanlar
                   (ad, soyad, tc, telefon, email, maas, cinsiyet, d_tarihi, baslama_tarihi, adres)
                   VALUES
                   (@ad, @soyad, @tc, @telefon, @email, @maas, @cinsiyet, @d_tarihi, @baslama_tarihi, @adres);";

    try
    {
        using var conn = Databases.GetConnection();
        using var cmd = new MySqlCommand(sql, conn);

        // Sorgu içindeki @ parametrelerine değerleri atıyoruz:
        cmd.Parameters.AddWithValue("@ad", calisan.Ad);
        cmd.Parameters.AddWithValue("@soyad", calisan.Soyad);
        cmd.Parameters.AddWithValue("@tc", calisan.Tc);
        cmd.Parameters.AddWithValue("@telefon", calisan.Telefon);
        cmd.Parameters.AddWithValue("@email", calisan.Email);
        cmd.Parameters.AddWithValue("@maas", calisan.maas);
        cmd.Parameters.AddWithValue("@cinsiyet", calisan.Cinsiyet);
        cmd.Parameters.AddWithValue("@d_tarihi", calisan.D_tarihi);
        cmd.Parameters.AddWithValue("@baslama_tarihi", calisan.Baslama_tarihi);
        cmd.Parameters.AddWithValue("@adres", calisan.Adres);

        conn.Open();
        int etkilenen = cmd.ExecuteNonQuery();
        return etkilenen > 0; // En az 1 satır eklendiyse true döner
    }
    catch (Exception ex)
    {
        hataMesaji = $"Veritabanı hatası: {ex.Message}";
        return false;
    }
}
```

💡 **`cmd.LastInsertedId` (Otomatik Oluşan Yeni ID'yi Alma):**

`INSERT` komutunu çalıştırdıktan hemen sonra, MySQL'in yeni eklenen kayda verdiği `AUTO_INCREMENT` ID numarasını almak için:

```
int etkilenen = cmd.ExecuteNonQuery();
long yeniCalisanId = cmd.LastInsertedId; // Yeni üretilen ID!
```

### B) Kayıt Güncelleme ve Silme (`UPDATE` & `DELETE`)

**Kayıt Güncelleme (`UPDATE`)**

```
string sql = @"UPDATE calisanlar
               SET maas = @maas, adres = @adres
               WHERE id = @id;";

using var conn = Databases.GetConnection();
using var cmd = new MySqlCommand(sql, conn);
cmd.Parameters.AddWithValue("@maas", yeniMaas);
cmd.Parameters.AddWithValue("@adres", yeniAdres);
cmd.Parameters.AddWithValue("@id", calisanId);

conn.Open();
bool basarili = cmd.ExecuteNonQuery() > 0;
```

**Kayıt Silme (`DELETE`)**

```
string sql = "DELETE FROM calisanlar WHERE id = @id;";

using var conn = Databases.GetConnection();
using var cmd = new MySqlCommand(sql, conn);
cmd.Parameters.AddWithValue("@id", calisanId);

conn.Open();
int silinen = cmd.ExecuteNonQuery();
bool basarili = silinen > 0;
```

## 6. Veritabanından Veri Okuma ve Listeleme (R)

### A) Tek Değer / Varlık Kontrolü (`SELECT COUNT(*)` + `ExecuteScalar`)

Bir çalışanın kayıtlı olup olmadığını (TC veya kullanıcı adı var mı?) denetlerken bütün satırı çekmeye gerek yoktur. Sadece adeti (sayıyı) çekmek en performanslı yoldur:

```
public bool TcVarMi(string tc)
{
    string sql = "SELECT COUNT(*) FROM calisanlar WHERE tc = @tc;";

    using var conn = Databases.GetConnection();
    using var cmd = new MySqlCommand(sql, conn);
    cmd.Parameters.AddWithValue("@tc", tc);

    conn.Open();
    // ExecuteScalar ilk hücreyi 'object' döner, Convert ile int'e çevrilir:
    int adet = Convert.ToInt32(cmd.ExecuteScalar());
    return adet > 0; // 0'dan büyükse bu TC zaten kayıtlıdır
}
```

### B) Çok Satırlı Veri Okuma & Model Listesi Doldurma (`MySqlDataReader`)

Veritabanındaki satırları tek tek okuyup `List<Calisan>` nesne listesine dönüştürmek için `while(dr.Read())` akışı kullanılır:

```
public List<Calisan> TumCalisanlariGetir()
{
    List<Calisan> liste = new List<Calisan>();
    string sql = "SELECT * FROM calisanlar ORDER BY ad ASC;";

    using var conn = Databases.GetConnection();
    using var cmd = new MySqlCommand(sql, conn);

    conn.Open();
    using var dr = cmd.ExecuteReader();
    while (dr.Read()) // Her satır için döner
    {
        Calisan c = new Calisan();
        c.Fill(dr); // 👈 Modelin içindeki Fill() metodu tek satırda doldurur!
        liste.Add(c);
    }
    return liste;
}
```

### C) WinForms İçin `DataTable` & `DataGridView` Veri Bağlama

Tüm veritabanı tablosunu tek satırda çekip formdaki `DataGridView` tablosuna dökmek istediğinizde **`MySqlDataAdapter`** en pratik çözümdür:

```
public DataTable CalisanTablosuGetir()
{
    DataTable dt = new DataTable();
    string sql = "SELECT id, ad, soyad, telefon, yetki, maas FROM calisanlar;";

    using var conn = Databases.GetConnection();
    using var adapter = new MySqlDataAdapter(sql, conn);

    // DataAdapter bağlantıyı kendisi açar, tabloyu doldurur ve kapatır (conn.Open() gerekmez):
    adapter.Fill(dt);
    return dt;
}

// Form İçindeki Kullanımı (Tek Satır):
dataGridView1.DataSource = calisanRepo.CalisanTablosuGetir();
```

## 7. İleri Düzey İpuçları, NULL Yönetimi ve Hata Yakalama

### 1. NULL Değerleri Güvenle Okuma

Veritabanında `NULL` olabilecek sütunlar (örn: `foto`) düz okunursa program çöker. `as string` ile tek satırda çözülür:

```
// En temiz ve modern yol:
Foto = reader["foto"] as string;

// Veya klasik kontrol:
if (!reader.IsDBNull(reader.GetOrdinal("foto")))
    Foto = reader.GetString("foto");
else
    Foto = null;
```

### 2. `DBNull.Value` ile NULL Yazma

C#'taki `null` değişkeni MySQL parametresine yazarken `DBNull.Value` verilmelidir:

```
if (string.IsNullOrEmpty(calisan.Foto))
    cmd.Parameters.AddWithValue("@foto", DBNull.Value);
else
    cmd.Parameters.AddWithValue("@foto", calisan.Foto);
```

### 3. MySQL Hata Kodları (Duplicate Entry)

Aynı TC veya e-posta ikinci kez eklenmeye çalışıldığında MySQL **1062** hata kodunu fırlatır:

```
catch (MySqlException ex)
{
    if (ex.Number == 1062)
        MessageBox.Show("Bu TC veya E-posta zaten kayıtlı!");
    else
        MessageBox.Show($"Veritabanı Hatası: {ex.Message}");
}
```

### 4. Transaction (İşlem Güvenliği)

Birden çok SQL adımının ya hep birlikte başarılı olması ya da hiçbirinin çalışmaması için kullanılır:

```
using var trans = conn.BeginTransaction();
try {
    cmd1.Transaction = trans; cmd1.ExecuteNonQuery();
    cmd2.Transaction = trans; cmd2.ExecuteNonQuery();
    trans.Commit(); // Her şey tamsa onayla!
} catch {
    trans.Rollback(); // Hata olursa her şeyi geri al!
}
```

📋 **5 Altın Veritabanı Kuralı:**

1. **Her bağlantıyı using ile açın:** Açık kalan bağlantılar MySQL havuzunu kilitler.
2. **Asla string birleştirme (`+`) ile SQL yazmayın:** Her zaman `@parametre` kullanın (SQL Injection koruması).
3. **Metot sorumluluğunu doğru seçin:** Değişiklik için `ExecuteNonQuery`, tek değer için `ExecuteScalar`, liste için `ExecuteReader`.
4. **Bağlantıyı olabildiğince geç açıp erken kapatın:** Sadece sorgu anında açın, form açılışında sürekli açık tutmayın.
5. **Katman disiplinini koruyun:** SQL sorgularını formların (UI) içine değil, yalnızca **Repository** sınıflarına yazın.